2. (4 Marks) In the knapsack problem, a container is given with capacity C and a set I of n items, where each item is described by its weight and value: $I = \{(w_1, v_1), (w_2, v_2), \cdots, (w_n, v_n)\}$. The optimal solution to this problem is a subset I' of I that maximizes $\sum_{(w,v)\in I'} v$ such that $\sum_{(w,v)\in I'} w \leq C$.

One of your friends is trying to model this problem as a combinatorial search problem that can be solved with stochastic hill climbing with random restarts. Recall that, in order to do so, they need to define the candidate solutions, the neighborhood function, which receives a candidate solution and returns a set of candidates, and the probability distribution over the neighbors of a candidate. Your friend writes down the following example to help you better understand the problem.

Example. Let $I = \{(10, 20), (5, 8), (8, 10), (3, 3)\}$ and $C = 12$. The solution $\{(8, 10)\}$ has the value of 10, while the solution $\{(5, 8), (3, 3)\}$ has the value of 11. Note that the solution $\{(10, 20), (5, 8)\}$ is infeasible because the sum of the weights is greater than C $(10 + 5 > 12)$.

Your friend told you that their implementation does not find good solutions to the knapsack problem. In fact, their approach tends to return solutions that are infeasible. They are pretty sure that it is a "conceptual bug" rather than an "implementation bug". That is, the problem is in how they modeled the problem. As an initial candidate, they use a random permutation of items in I. The neighborhood function generates neighbors by swapping two items in the vector in which the candidate is represented.

Explain what is the conceptual bug in their approach. Then, explain how to fix it.

The candidate space only considers permutations of the items, while it should consider subsets of items. The problem can be fixed in three different ways; any of the following is correct.

(a) Fixing both the initial candidate function and the neighborhood function: The initial candidate function generates a subset of items and the neighborhood function allows for the generation of neighbors with more or fewer items than the current candidate.

(b) Fixing only the neighborhood function: the neighborhood function allows for the generation of neighbors with more or fewer items than the current candidate.

(c) Fixing only the initial candidate function: the initial candidate function generates a **random** subset of the items. The initial subset needs to be random so that the algorithm is able to search over subsets of different sizes in each restart of the search.

3. (4 Marks) One of your classmates is confused about the heuristic for selecting variables, Minimum Remaining Value (MRV), and the heuristic for selecting values, Least Constraining Value (LCV), for solving Constraint Satisfaction Problems with Backtracking search. According to your classmate, these heuristics seem to be contradictory: since MRV selects the variable with the smallest domain, it is selecting the most constrained variable; on the other hand, LCV selects the value that will constrain the least the problem. Explain in a few sentences why the reasoning of your friend is flawed by explaining the intuition behind these two heuristics. That is, explain why they tend to work well in practice.

The answer to this question has two parts. One explaining the intuition behind MRV and another explaining the intuition behind LCV.

(a) The heuristics are not contradictory because they both attempt to reduce the size of the search tree. MRV uses the branching factor of a node as an approximation to the size of the tree rooted at that node. Variables with smaller domains have a smaller branching factor.

(b) If the partial assignment does not lead to a solution (i.e., it is a nogood), then the order in which we search over values does not matter. This is because we need to search over all values no matter what. The size of the search tree can be smaller if there is a solution in a given subtree. Choosing the least-constraining value can help determine the branch of the tree that contains the solution, thus avoiding branches that do not lead to a solution.

c) (2 Marks) Explain in two short sentences the difference in value between h^* and $h^{\max}$. That is why does $h^{\max}$ lose information with respect to h^*?

It loses information due to the delete relaxation effect: once something becomes true, it wil remain true. And because it assumes that each sub-goal can be solved independently.

d) (2 Marks) What if we increased the number of packages in city d that needs to be delivered in city a (everything else is held constant in the problem), would the $h^{\max}$-value increase, remain the same, or decrease? It would remain the same.

e) (2 Marks) What if we increased the number of packages in city d that needs to be delivered in city a (everything else is held constant in the problem), would the h^{add}-value increase, remain the same, or decrease? It would increase.

f) (2 Marks) Explain your answers to letters (d) and (e) above.

$h^{\max}$ remains the same because we add goal conditions that cost as much as the most expensive goal conditions of the original problem; the maximum of all these values will remain the same. h^{add} will increase because the heuristic sums the costs of all sub-goals, when solved independently, so that must increase the estimate of the cost-to-go of the heuristic.

b) (3 Marks) Explain why $h^{\max}$ returns ∞ for its initial state?

We cannot move any discs in this problem due to the precondition "smaller ?to ?disc". Propositions of this type are not true in the initial state and there is no action to make them true, so the value of the goal propositions are initialized with ∞ and they will remain ∞ during the $h^{\max}$ computation.

Paper 2

1. (5 Marks) Best-first search algorithms use a cost function f to sort the nodes in the OPEN list and, in every iteration, they expand the node with the cheapest f-value in OPEN. Recall that for A*, $f(n) = g(n) + h(n)$, and for WA*, $f(n) = g(n) + w \cdot h(n)$, with $w > 1$. In this question, you should consider $w = 10$. Consider the search problem shown in the figures below. The initial state is given by the cell marked with the letter "i" and the goal is marked with the letter "g". The agent can move up, down, left, or right, and each action costs 1.0. Black cells denote walls, which the agent cannot traverse.

A*

9	○ 7	7	
	○ 5	° ○ 5	7
	○ 5	○ 5	○ 7
			○ 7
	○ 7	○ 7	○ 7
	° ○ 7	7	7

WA*

63	○ 52	61	
	○ 41	○ 50	61
	○ 32	○ 41	○ 52
			○ 43
	○ 16	○ 25	○ 34
	° ○ 7	16	25

Based on this description, answer the questions below. In case of ties, you should favor states with larger g-values; if the tie persists, then favor leftmost states on the grid. Assume the heuristic function used is Manhattan distance.

a) (2 Marks) Write in the cells of the grids the f-value of the states the algorithms **generate** while solving the problem. A cell will be empty only if the state it represents is not generated.

b) (2 Marks) Mark each cell representing an **expanded** state with the symbol ○.

c) (1 Mark) Do both algorithms find an optimal solution to this problem? (yes or no)

yes

2. (7 Marks) In the subset selection problem, you are given a set of items $S = \{s_1, s_2, \ldots, s_n\}$ and a positive integer $k < n$. The objective is to select exactly k items from S that maximize a given evaluation function f. This problem can be approached as an optimization problem solvable with local search algorithms such as hill climbing. To formulate it this way, you need to define the candidate solutions and a neighborhood function that receives a candidate solution and returns a set of neighbor candidates.

a) (2 Marks) Describe the set of candidate solutions to this problem.

All possible subsets of S with k items.

b) (3 Marks) Describe a neighborhood function to this problem.

A neighborhood function generates a number of neighbors of a candidate C by following these two steps (the execution of these two steps generates one neighbor).

i. Randomly remove an element from C.

ii. Randomly add to C an element from $S \setminus C$.

c) (2 Marks) Which search algorithm would you use to solve this problem if you had plenty of tim to run the algorithm (e.g., a few days of computation). There is no need to justify your answe

Hill climbing with random restarts.

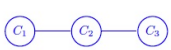
3. (8 Marks) A university needs to schedule courses into available time slots. Each course must be assigned exactly one time slot, and certain pairs of courses cannot be scheduled simultaneously because some students take both courses. You should assume that the university has plenty of lecture rooms available, so you only need to be concerned with the time of the day when the courses will be delivered. Considering this timetabling problem, answer the following questions.

a) (3 Marks) Formulate the timetabling problem described above as a Constraint Satisfaction Problem (CSP). Recall that you need to define the variables, domains, and constraints of the problem.

Each course C_i is a variable and the domains are the time slots t_j available. We will have constraints of the form: $(C_j, C_k, \neq)$ if there is at least one student enrolled in both C_j and C_k.

b) (3 Marks) Give a small example of the problem you have formulated. Your example should have 3 courses and 2 time slots and you should include the variables, domains, and a graph representing the constraints of the problem.

The set of courses $\{C_1, C_2, C_3\}$ with time slots (domains) $\{t_1, t_2\}$. As for constraints, there are students enrolled in C_2 who are also enrolled in C_1 and C_3. There are no students enrolled in both C_1 and C_3.



c) (2 Marks) Briefly explain why assigning a time slot to one course can simplify the possible time slots for other courses.

Once a time slot t is assigned to a course c, all courses connected to c can have t removed from their domain. This is because courses involved in a binary constraint cannot be scheduled at the same time.

Based on this description, answer the questions below. In case of ties, you should favor states with larger g-values; if the tie persists, then favor leftmost states on the grid. In the questions below, you should assume that the heuristic function used is Manhattan distance.

a) (2 Marks) Write in the cells of the grids the f-value of the states the algorithms **generate** in search while solving the problem. A cell will be empty only if the state it represents is not generated in search.

b) (1 Mark) Mark each cell representing an **expanded** state with the symbol ○.

c) (1 Mark) Do all algorithms find an optimal solution to this problem? (yes or no) Yes.

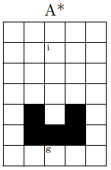
d) (1 Mark) Which algorithm does the least number of expansions? GBFS.

e) (1 Mark) If the cell right above the goal cell had no wall (i.e., it was an empty cell), which algorithms would find the optimal solution? A*, WA*, and GBFS.

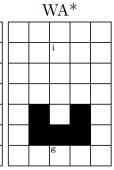
f) (1 Mark) Consider now the case in which the agent can traverse wall cells at the cost of 10.0 to enter a cell with a wall and at the regular cost of 1.0 to leave a cell with a wall. Which algorithms return a **sub-optimal solution**? You should assume that the walls are as shown in the images. WA* and GBFS.

1. (7 Marks) Best-first search algorithms use a cost function f to sort the nodes in the OPEN list and, in every iteration, they expand the node with the cheapest f-value in OPEN. Recall that for A*, $f(n) = g(n) + h(n)$, for WA*, $f(n) = g(n) + w \cdot h(n)$, with $w > 1$. In this question, you should consider $w = 10$. Finally, for Greedy-Best First Search, $f(n) = h(n)$. Consider the search problem shown in the figures below. The initial state is given by the cell marked with the letter "i" and the goal is marked with the letter "g". In this problem, the agent can move up, down, left, or right and every action costs 1.0. Black cells denote walls, which the agent cannot traverse.

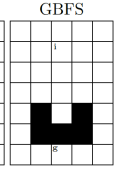
A*



WA*



GBFS



	9	○ 7	9	
9	○ 7	° ○ 5	○ 7	9
9	○ 7	○ 5	○ 7	9
○ 9	○ 7	○ 5	○ 7	9
○ 9		○ 5		
○ 9				
○ 9	○ 9	° ○ 9		

		61			
	61	° ○ 50	61		
63	○ 52	○ 41	○ 52	63	
○ 54	○ 43	○ 32	○ 43	54	
○ 45		○ 23			
○ 36					
○ 27	○ 18	° ○ 9			

		6		
	6	° ○ 5	6	
6	5	○ 4	5	
○ 5	○ 4	○ 3	○ 4	5
○ 4		○ 2		
○ 3				
○ 2	○ 1	° ○ 0		

4. (4 Marks) In the problem of Towers of Hanoi we have 3 pegs and a number of discs, all of different sizes. The discs are placed on the leftmost peg, sorted by size, with the largest disc at the bottom and the smallest at the top. The goal is to move all discs to the rightmost peg, while using the middle peg as an "auxiliary peg". While moving discs, one cannot place a larger disc on top of a smaller one. The image shows the initial (left) and goal (right) states of an instance of the problem with three discs.

Consider the following specification of the problem in PDDL, where we assume unit action costs.

```
(define (domain hanoi)
(:requirements :strips)
(:predicates (clear ?x)
             (on ?x ?y)
             (smaller ?x ?y))
(:action move
:parameters (?disc ?from ?to)
:precondition (and (smaller ?disc ?to)
                   (on ?disc ?from)
                   (clear ?disc)
                   (clear ?to))
:effect  (and (clear ?from)
              (on ?disc ?to)
              (not (on ?disc ?from))
              (not (clear ?to))) ))
```

a) (2 Marks) The instance p1, described below, has 3 discs on peg1 in the initial state and the goal is to move all discs to peg3. How would you modify the preconditions of the action move so you can solve p1 with a single action? Once you modify move, you will not necessarily follow the original rules of the puzzle. There is no need to justify your answer.

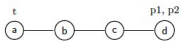
```
(define (problem p1)
(:domain hanoi)
(:objects peg1 peg2 peg3 d1 d2 d3 )
(:init  (smaller d1 d2) (smaller d1 d3) (smaller d2 d3)
        (smaller d1 peg1) (smaller d1 peg2) (smaller d1 peg3)
        (smaller d2 peg1) (smaller d2 peg2) (smaller d2 peg3)
        (smaller d3 peg1) (smaller d3 peg2) (smaller d3 peg3)
        (clear peg2) (clear peg3) (clear d1)
        (on d3 peg1) (on d2 d3) (on d1 d2) )
(:goal (and (on d3 peg3) (on d2 d3) (on d1 d2))))
```

Remove precondition `(clear ?disc)`

b) (2 Marks) How would you modify the initial state I of p1 such that both h^{add} and $h^{\max}$ would return ∞ for I? You should consider the original formulation of the problem, and not the one you modified in (a) above.

Remove the proposition `(smaller d3 peg3)` from I

5. (6 Marks) In the following logistic problem, we have four cities: a, b, c, d, a truck that starts in city a (denoted by t), and two packages initially located in city d (denoted by p1 and p2). The actions are drive(x, y), load(x, y), and unload(x, y). The preconditions and effects are what one expects: you can only load a package into the truck if both the package and the truck are in the same city; you can only unload a package from the truck if the package is in the truck. Once the truck is loaded, the package is no longer in the city, but in the truck; once a package is unloaded from the truck, the package is no longer in the truck, but in the city where it was unloaded. You can only load and unload one package at a time. You can assume that the truck is not limited on the number of packages it is able to carry at a given time. The truck can only drive between two cities if the cities are connect by a road (an edge in the graph below). All actions cost one. The initial state is $I = \{$(at t, a), (at, p1, d), (at, p2, d)$\}$ and the goal conditions are $G = \{$(at t, d), (at p1, a), (at p2, a)$\}$. That is, p1 and p2 must be moved from d to a and t from a to d.



Answer the following questions with respect to the above problem.

a) (2 Marks) What is the h^*-value of I? 13

b) (2 Marks) What is the h^+-value of I? 7

c) (2 Marks) Explain in one short sentence the difference in value between h^* and h^+ in the previous question. That is, why does h^+ lose information with respect to h^*?

It loses information due to the delete relaxation effect: once something becomes true, it will remain true.